第16课 枚举法（二）——多重循环与巧妙枚举

> 本讲义配合课堂例题讲解使用，代码可直接在 Dev-C++ 中运行验证。

五、课堂例题

例题1：鸡兔同笼（入门）

题目：笼子里有鸡和兔共 `n` 个头，`m` 只脚。输出鸡和兔各有多少只。如果无解输出 `No answer`。

输入样例：`35 94`
输出样例：`23 12`（23只鸡，12只兔）

方法一：双重循环（暴力版）

讲解思路：
- 两个循环分别枚举鸡和兔的数量
- 检查两个条件：头数之和 = n，脚数之和 = m
- 这是最直观的"枚举法两步走"：先列举，再筛选

```
#include <iostream>
using namespace std;

int main() {
    int n, m;
    cin >> n >> m;
    bool found = false;

    // 枚举鸡的数量
    for (int chicken = 0; chicken <= n; chicken++) {
        // 枚举兔的数量
        for (int rabbit = 0; rabbit <= n; rabbit++) {
            // 筛选：头数和脚数都要对
            if (chicken + rabbit == n && chicken * 2 + rabbit * 4 == m) {
                cout << chicken << " " << rabbit << endl;
                found = true;
            }
        }
    }

    if (!found) {
        cout << "No answer" << endl;
    }

    return 0;
}
```

方法二：单重循环（优化版）

讲解思路：

- 关键发现：知道鸡的数量 `a`，兔的数量就是 `n - a`
- 只需要一个循环枚举鸡，兔由鸡推出
- 只需检查一个条件：脚数是否等于 m

```
#include <iostream>
using namespace std;

int main() {
    int n, m;
    cin >> n >> m;
    bool found = false;

    for (int chicken = 0; chicken <= n; chicken++) {
        int rabbit = n - chicken;  // 用鸡推出兔
        if (chicken * 2 + rabbit * 4 == m) {
            cout << chicken << " " << rabbit << endl;
            found = true;
        }
    }

    if (!found) {
        cout << "No answer" << endl;
    }

    return 0;
}
```

对比引导：让学生分别运行两个版本，对比循环次数。暴力版循环 n×n 次，优化版只循环 n 次。当 n = 1000 时，暴力版要检查一百万次，优化版只要一千次——差了 1000 倍！

例题2：填运算符（进阶）

题目：在 `1 ○ 2 ○ 3 ○ 4` 的三个圆圈中填入 `+` 或 `-`，使得结果等于 4。输出所有方案。

输入：无
输出示例：

```
1+2-3+4=4
1-2+3+4=4   （虚拟示例，以实际计算为准）
```

讲解思路：
- 3 个运算符位置，每个位置 2 种选择，共 2×2×2 = 8 种组合
- 用 0 表示 `+`，用 1 表示 `-`
- 三重循环枚举三个运算符
- 计算结果，筛选等于 4 的方案

```
#include <iostream>
using namespace std;

int main() {
    // op1, op2, op3 分别表示三个运算符
    // 0 表示 +, 1 表示 -
    for (int op1 = 0; op1 <= 1; op1++) {
        for (int op2 = 0; op2 <= 1; op2++) {
            for (int op3 = 0; op3 <= 1; op3++) {
                // 计算结果
                int result = 1;  // 从第一个数开始

                if (op1 == 0) result = result + 2;
                else          result = result - 2;

                if (op2 == 0) result = result + 3;
                else          result = result - 3;

                if (op3 == 0) result = result + 4;
                else          result = result - 4;

                // 筛选
                if (result == 4) {
                    // 输出算式
                    cout << "1";
                    if (op1 == 0) cout << "+"; else cout << "-";
                    cout << "2";
                    if (op2 == 0) cout << "+"; else cout << "-";
                    cout << "3";
                    if (op3 == 0) cout << "+"; else cout << "-";
                    cout << "4";
                    cout << "=" << result << endl;
                }
            }
        }
    }

    return 0;
}
```

运行结果：

```
1+2-3+4=4
1-2+3+4=4  （虚拟示例，以实际运行为准）
```

教学小贴士：让学生先手动列出 8 种组合，在纸上算出结果，再和程序输出对比，增强对"计算机帮我们一个个试"的体感。

例题3：数字拆分（提高）

题目：输入正整数 `n`（n ≤ 20），把 `n` 拆分成三个正整数之和（要求 a ≤ b ≤ c），输出所有拆分方案和方案总数。

输入样例：`7`
输出样例：

```
1 1 5
1 2 4
1 3 3
2 2 3
4
```

讲解思路：
- 要求 a ≤ b ≤ c，避免重复
- a 的范围：1 到 n/3（因为 a 是最小的，三个 a 不超过 n）
- b 的范围：a 到 (n-a)/2（因为 b 是中间的，b 和 c 分剩下的，b ≤ c）
- c 不用枚举，直接 c = n - a - b
- 最后检查 c ≥ b（其实由 b 的范围已经保证了）

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cin >> n;
    int count = 0;

    for (int a = 1; a <= n / 3; a++) {                // a 是最小的
        for (int b = a; b <= (n - a) / 2; b++) {      // b 从 a 开始，不超过剩余的一半
            int c = n - a - b;                          // c 直接算出来
            cout << a << " " << b << " " << c << endl;
            count++;
        }
    }

    cout << count << endl;

    return 0;
}
```

优化讲解：

> 如果不加 a ≤ b ≤ c 的限制，三重循环要跑 n×n×n 次。加了限制后，循环次数大大减少。而且 c 不用循环，直接算出来，又省了一层！
> 这就是枚举优化的秘诀：**能算出来的就不要去试，能缩小范围的就尽量缩小。**

附：例题源代码与运行结果

例题 2：Fill in Operators

```
// Place + or - in: 1 O 2 O 3 O 4 = 4
// Enumerate all 3 operator positions (each can be + or -).
// Use 0 for +, 1 for -.

/*
#include <iostream>
using namespace std;

int main() {
    // op1, op2, op3 represent three operators
    // 0 means +, 1 means -
    for (int op1 = 0; op1 <= 1; op1++) {
        for (int op2 = 0; op2 <= 1; op2++) {
            for (int op3 = 0; op3 <= 1; op3++) {
                // Calculate the result starting from the first number
                int result = 1;

                if (op1 == 0) result = result + 2;
                else          result = result - 2;

                if (op2 == 0) result = result + 3;
                else          result = result - 3;

                if (op3 == 0) result = result + 4;
                else          result = result - 4;

                // Filter: check if result equals 4
                if (result == 4) {
                    cout << "1";
                    if (op1 == 0) cout << "+"; else cout << "-";
                    cout << "2";
                    if (op2 == 0) cout << "+"; else cout << "-";
                    cout << "3";
                    if (op3 == 0) cout << "+"; else cout << "-";
                    cout << "4";
                    cout << "=" << result << endl;
                }
            }
        }
    }

    return 0;
}
*/
```

Expected Output

```
1-2+3+4=4 (virtual example, verify by running)
1+2-3+4=4
```

例题 3：Split a Number into Three Parts

```
// Input n, split it into a + b + c (a <= b <= c, all positive).
// Output all solutions and the total count.

/*
#include <iostream>
using namespace std;

int main() {
    int n;
    cin >> n;
    int count = 0;

    for (int a = 1; a <= n / 3; a++) {                // a is the smallest
        for (int b = a; b <= (n - a) / 2; b++) {      // b starts from a, up to half of 
            int c = n - a - b;                          // c is computed directly
            cout << a << " " << b << " " << c << endl;
            count++;
        }
    }

    cout << count << endl;

    return 0;
}
*/
```

Sample Input

```
7
```

Expected Output

```
1 1 5
1 2 4
1 3 3
2 2 3
4
```